

CFD Assignment 3 on Unsteady Thermal Diffusion on Circular Disk

- By Souvik Sarkar (22ME10083)

Derivation and Accuracy Analysis:

CFD Assignment 3 - 22ME10083

Assumption: Gray Cast Iron

- i) Thermal Conductivity (k) = $53 \text{ W/m}\cdot\text{K}$.
- ii) Specific heat (c_p) = $460 \text{ J/kg}\cdot\text{K}$
- iii) Density (ρ) = 7200 kg/m^3

Unsteady Heat conduction Equation in polar coordinate w/o source term.

$$\rho c_p \frac{\partial T}{\partial t} = \nabla \cdot (k \nabla T) \quad \text{assume, } k \text{ is constant,}$$

$$\Rightarrow \left[\frac{1}{r} \frac{\partial T}{\partial t} = \frac{\partial^2 T}{\partial r^2} + \frac{1}{r} \frac{\partial T}{\partial r} + \frac{1}{r^2} \frac{\partial^2 T}{\partial \theta^2} \right]$$

Subjected to $T(r_{out}, \theta, z) \big|_{180^\circ} = T_{u_0} = 70^\circ \text{C}$; $T(r_{in}, \theta, z) \big|_{360^\circ} = T_{B_0} = 30^\circ \text{C}$

Assume, $T(r_{out}, \theta = 0/360, z) = \frac{70+30}{2} = 50^\circ \text{C}$, $T(r_{out}, \theta = 180, z) = \frac{70+30}{2} = 50^\circ \text{C}$

For using FTCS or Euler Explicit Scheme,

$$\gamma = \frac{\Delta x}{\Delta t} = \frac{\alpha \Delta t}{\Delta x^2} < \frac{1}{4} \Rightarrow \Delta t < \frac{1}{4} \Delta x^2 \Rightarrow \boxed{\Delta t < 15622.6}$$

Now, characteristic length $l_c = r_{out} - r_{in} = 15 \text{ cm} = 0.15 \text{ m}$.

$$\tau_c \approx \frac{l_c^2}{\alpha} \approx 1406.25 \text{ sec} \rightarrow \text{needed to attain steady state}$$

Now, FTCS scheme has a order of accuracy $O(\Delta x^2, \Delta t)$

let, $\Delta x = R_{avg} \Delta \theta$, $R_{avg} = 0.2 \text{ m}$

and we have total 31 grid point along $N_r=31$ each radial direction, then

$$\Delta x = \frac{R_{out} - R_{in}}{30} = \frac{0.3 - 0.15}{30} = 5 \times 10^{-3} \text{ m}$$

So, $\Delta t < 15622.6 \times (5 \times 10^{-3})^2 < 0.3905 \text{ seconds}$

we choose $\Delta t \Rightarrow 0.2 \text{ seconds}$

And, the Angular discrete grid separation $\Delta \theta = \frac{5 \times 10^{-3}}{0.2} = 0.025 \text{ rad}$

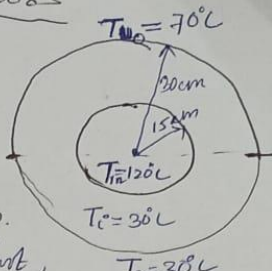
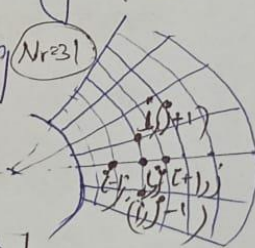
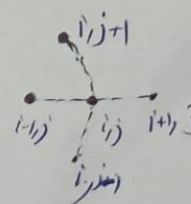
So, the total No. Angular Grid point $N_\theta = 251$

$$N_\theta = 31$$

Now, order of accuracy for FTCS

$$(\Delta R)^2 = \left(\frac{\Delta x}{l_c} \right)^2 = 1.11 \times 10^{-3} \quad \Delta t = \frac{\alpha \Delta t}{l_c^2} = 1.13 \times 10^{-5}$$

FTCS has order of accuracy $O(\Delta R^2) = 1.11 \times 10^{-3}$.

Formulation & Discretization using FTCS or Euler Explicit Scheme:

First it is non dimensionalized and make it vector using pointer

Let, us make the G.P.D as non-dimensional Equation

$$\phi = \frac{T - T_c}{T_{in} - T_c} \quad T = T_{in} \rightarrow \phi = 1 \quad | \quad T = T_c \rightarrow \phi = 0$$

(finite positive no.)

$$R = \frac{r - R_{in}}{R_{out} - R_{in}} \quad \Delta R = \frac{\Delta r}{R_{out} - R_{in}} = \frac{\Delta r}{L_c} \quad (0 < R < 1)$$

$$\psi = \frac{\theta}{2\pi} \Rightarrow \Delta \psi = \frac{\Delta \theta}{2\pi} \quad (0 < \psi < 1) \quad | \quad L = \frac{L_c^2}{\alpha \tau}$$

Now, $R=0$, ($r=R_{in}$) $\phi(R=0) = \frac{T_{in} - T_c}{T_{in} - T_c} = 1 \rightarrow$ BC inside of the cylinder.

Now, $R=1$ ($r=R_{out}$) $\phi(R=1, 0.5 < \psi < 1) = \frac{70 - 30}{120 - 30} = 0.4444$ - does outside here given

$$\phi(R=1, 0.5 < \psi < 1) = \frac{30 - 30}{120 - 30} = 0$$

and $\phi(R=1, \psi=0, \tau) = \frac{50 - 30}{120 - 30} = 0.555 = \phi(R=1, \psi=1, \tau)$

FTCS Discretized, Equation, $\phi(R, \psi, \tau=0) = 0$

$$\frac{(\phi_{i,j}^{n+1} - \phi_{i,j}^n)}{\Delta \tau} = \frac{1}{R_i^*} \left(\frac{\phi_{i+1,j}^n - \phi_{i-1,j}^n}{2\Delta R} \right) + \frac{1}{R_i^*} \left(\frac{\phi_{i,j+1}^n - 2\phi_{i,j}^n + \phi_{i,j-1}^n}{\Delta \psi^2} \right)$$

with an error of $O(\Delta R^2, \Delta \tau)$

$$C_r = \frac{\alpha \Delta \tau}{\Delta R^2} = \frac{\Delta \tau}{R_i^* \Delta R} \quad | \quad C_i = \frac{\Delta \tau}{R_i^* \Delta \psi^2}$$

$$\phi_{i,j}^{n+1} = \phi_{i,j}^n (1 - 2C_r - 2C_i) + \phi_{i+1,j}^n (C_r - C_i) + \phi_{i-1,j}^n (C_r + C_i) + \phi_{i,j+1}^n (C_i) + \phi_{i,j-1}^n (C_i)$$

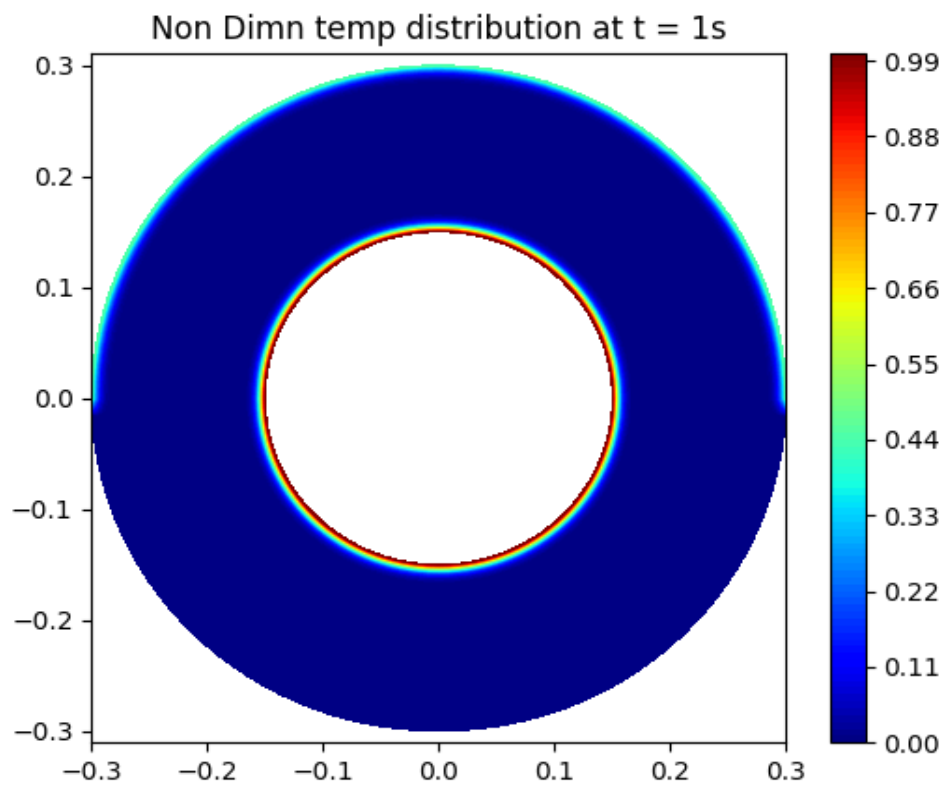
Using this pointer, $ip1(i,j) = (j-1) \times (N_{\psi}) + i$ we can form the matrix

where $R_i^* = \frac{R_{in}}{R_{out} - R_{in}} + R_i \Rightarrow 1 + R_i$

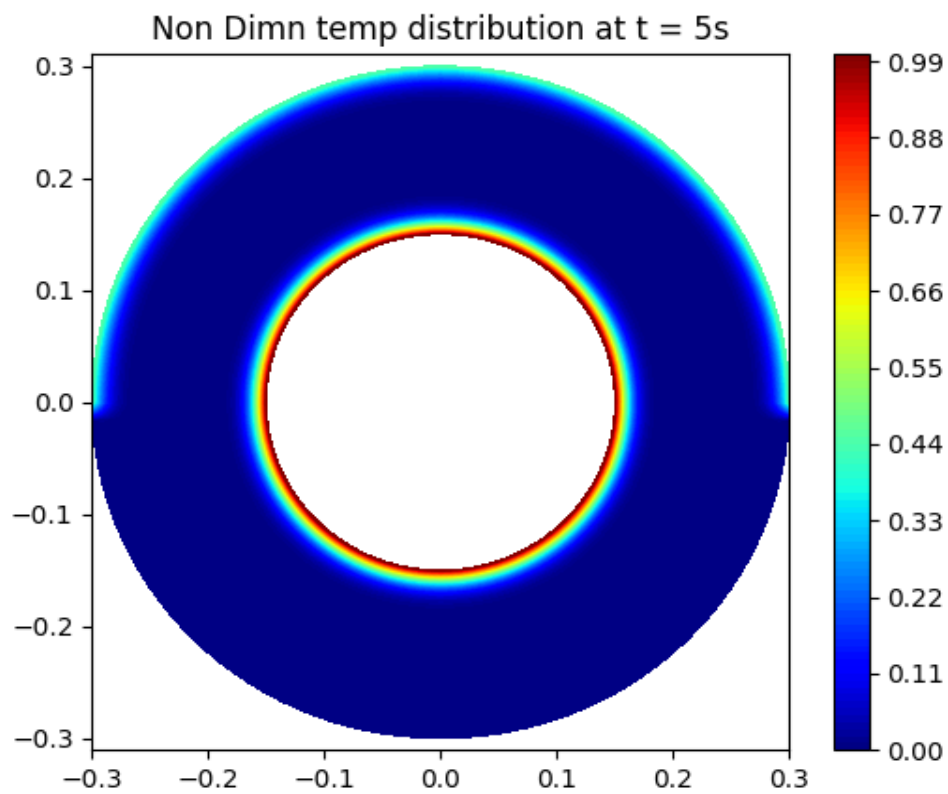
$$\phi_K^{n+1} = (1 - 2C_r - 2C_i) \phi_K^n + (C_r + C_i) \phi_{K+1}^n + (C_r - C_i) \phi_{K-1}^n + C_i \phi_{K,N_{\psi}}^n + C_i \phi_{K,N_{\psi}-1}^n$$

$$\begin{Bmatrix} \phi_1 \\ \phi_2 \\ \vdots \\ \phi_K \\ \vdots \\ \phi_{N_{\psi} \times N_{\psi}} \end{Bmatrix}^{n+1} = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ (C_r - C_i) & (1 - 2C_r - 2C_i) & (C_r + C_i) & \dots & 0 \\ 0 & \dots & C_i & \dots & (C_r - C_i) & (1 - 2C_r - 2C_i) & (C_r + C_i) & \dots & 0 \\ 0 & \dots & 0 & \dots & C_i & \dots & (C_r - C_i) & (1 - 2C_r - 2C_i) & (C_r + C_i) & \dots & 0 \\ 0 & \dots & 0 & \dots & 0 & \dots & 0 & \dots & 0 & \dots & 1 \end{bmatrix} \begin{Bmatrix} \phi_1 \\ \vdots \\ \phi_K \\ \vdots \\ \phi_{N_{\psi} \times N_{\psi}} \end{Bmatrix}^n$$

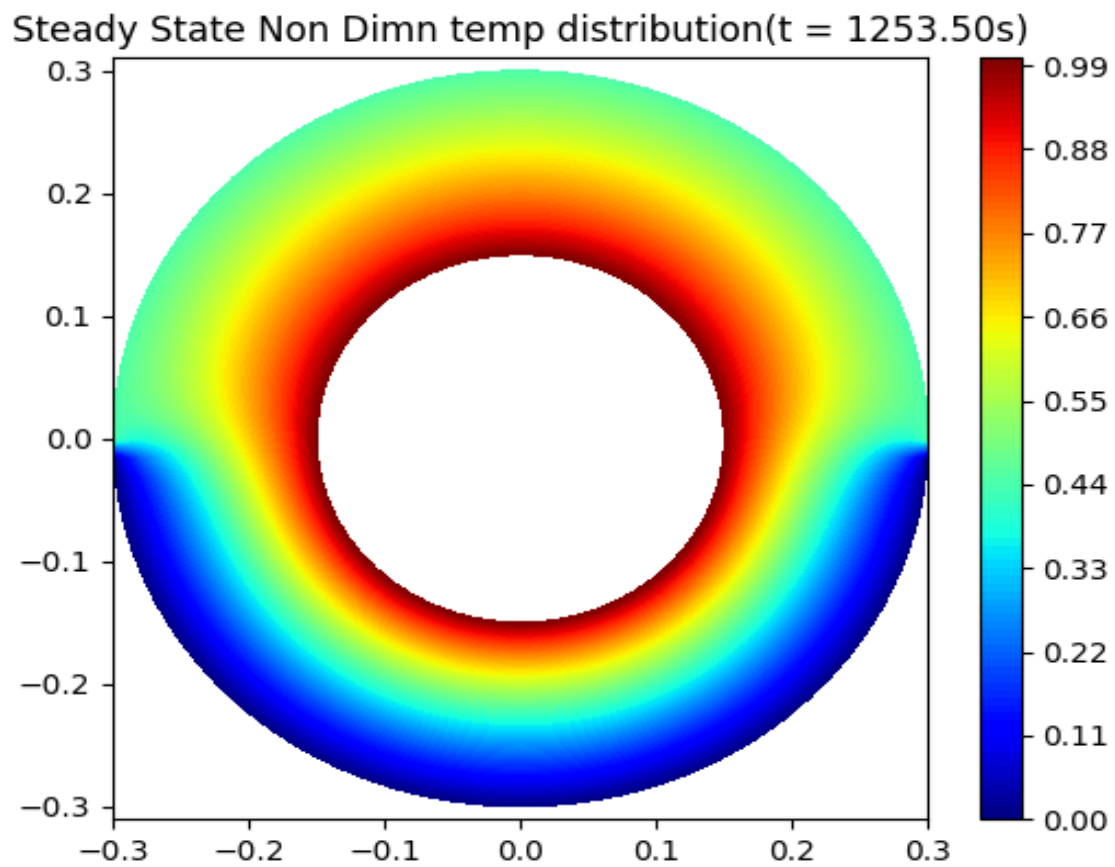
Results and Discussion : 1. Snap Shot at 1s :



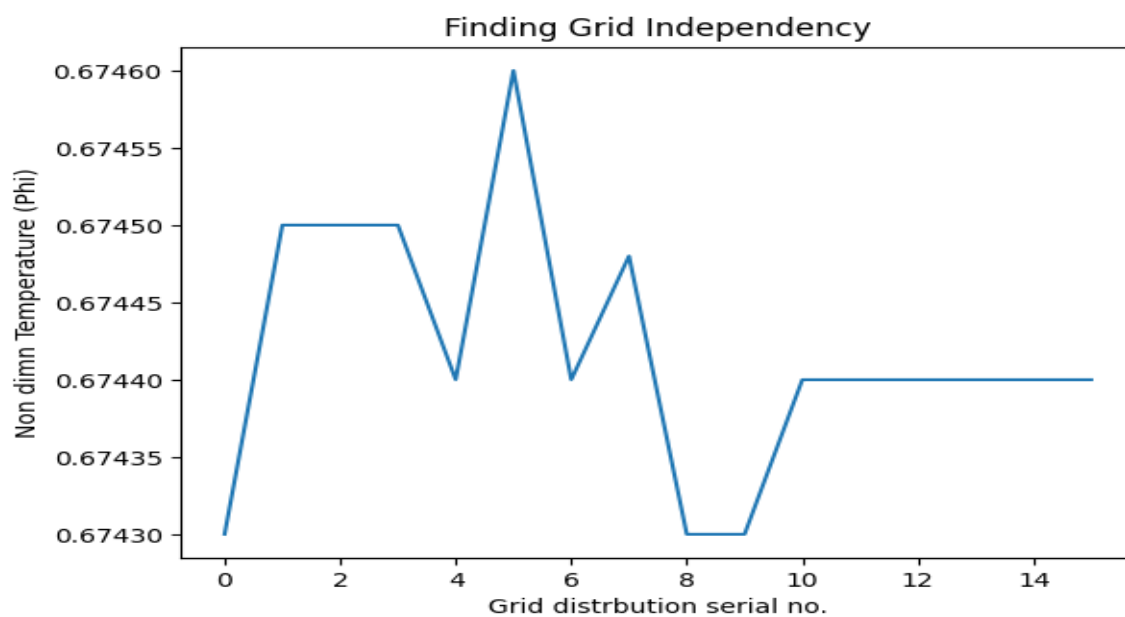
2. Snapshot at 5 seconds :



3. Steady state plot :



4. Grid Independency :



**** Grid Independency attain at '10' -> $N_r = 41$ and $N_{\theta} = 121$ resolution**

5. Working Code Snippet :

```
# -----  
#                               CFD Assignment 3 by 22ME10083  
#-----  
## NON DIMENTIONAL APPROACH  
  
import numpy as np  
import matplotlib.pyplot as plt  
  
# 1. Material Properties for Cast Iron  
k = 53.0           # Thermal conductivity (W/m.K)  
rho = 7200.0       # Density (kg/m^3)  
cp = 460.0         # Specific heat (J/kg.K)  
alpha = k / (rho * cp) # Thermal Diffusivity ( m2/s)  
  
# Geometric property  
  
ri, ro = 0.15, 0.30  
Lc = ro - ri  
ravg = 0.25  
  
## Analysis of stability condition  
# assume :  
Nr = 41;  
dr = (ro - ri) / (Nr - 1)  
print(f"dr = {dr}")  
  
#dt = 0.2 * (1/alpha) * (1 / (1/dr**2 + 1/(ri*dtheta)**2))  
dt_required = 0.25 * (1/alpha) * dr**2;  
print(f"dt_required < {dt_required} so we choose 0.1s")  
dt = 0.1;  
  
# As create a huge no. of iteration so excluded  
#dtheta = dr/ravg;  
#print(f"dtheta = {dtheta}")  
  
Ntheta = 121;  
dtheta = 2*np.pi/(Ntheta-1);  
print(f"Ntheta = {Ntheta}")  
  
## Approaximating the steady state time  
t_st = Lc**2/alpha;  
print(f"time for steady state approaximately (lumped mass assumption) =  
{t_st}")  
  
# 3. Initialization and Boundary Conditions  
# domain variable
```

```

r = np.linspace(ri, ro, Nr)
theta = np.linspace(0, 2 * np.pi, Ntheta)
time = np.linspace(0, t_st, int(t_st/dt));
## Given temps

T_inner = 120
T_outer_up = 70
T_outer_bottom = 30
T_initial = 30
# domain temps
T = T_initial * np.ones((Nr, Ntheta))
T[0, :] = T_inner;
# Outer radius boundary conditions
for j in range(Ntheta):
    if 0 <= theta[j] <= np.pi:
        T[Nr-1, j] = T_outer_up # Upper half
    else:
        T[Nr-1, j] = T_outer_bottom # Lower half

# Coordinates for plotting
Theta, R = np.meshgrid(theta, r)
X = R * np.cos(Theta)
Y = R * np.sin(Theta)

#####----- Making Non dimentional -----
Phi = (T - T_initial)/(T_inner - T_initial);
R = (r-ri)/(ro-ri);
Theta = theta/(2*np.pi);
Tau = alpha*time/Lc**2;

dR = dr/Lc;
dTheta = dtheta/2*np.pi;
dTau = dt*alpha/Lc**2;
## tolerance
tol = 1e-7;
Tau_cur = 0.0;
steady = False;
t_plotted = {1: False, 5: False} # Initialize t_plotted here
## loop for the main computation of the derivatives
while not steady:
    Phi_old = Phi.copy();
    Phi_new = Phi.copy();
    # main logic
    for i in range(1, Nr-1):
        for j in range(Ntheta-1):

```

```

        j_prev = (Ntheta - 2) if j == 0 else (j - 1)
        j_next = j + 1
        # Spatial derivatives
        #d2Phi_dR2 = (Phi[i+1, j] - 2*Phi[i, j] + Phi[i-1, j]) /
dR**2
        #dPhi_dR = (Phi[i+1, j] - Phi[i-1, j]) / (2*dR)
        #d2Phi_dTheta2 = (Phi[i, j_next] - 2*Phi[i, j] + Phi[i,
j_prev]) / dTheta**2

        #
        Cr = dTau/dR**2;
        Ci = dTau/(2*(ri/Lc + R[i])*dR);
        Di = dTau/(((ri/Lc + R[i])**2)*dTheta**2);

        Phi_new[i, j] = (1-2*Cr-2*Di)*Phi[i, j] + (Cr+Ci)*Phi[i+1,
j] + Phi[i-1, j]*(Cr-Ci) + Di*Phi[i, j_next] + (Di)*Phi[i, j_prev] ;

    # as Theta_0 is same as Theta_1 ( 0 == 2*pi)
    Phi_new[:, Ntheta-1] = Phi_new[:, 0];
    Phi = Phi_new;
    Tau_cur = Tau_cur + dTau; #*alpha/Lc**2;
    t_cur = (Tau_cur* Lc**2 )/alpha ;

    # Plot snapshot at t =1s and 5s
    # Plotting check
    for t_mark in [1, 5]:
        if t_cur >= t_mark and not t_plotted[t_mark]:
            plt.figure(figsize=(6, 5))
            cp = plt.contourf(X, Y, Phi, 100, cmap='jet')
            plt.colorbar(cp)
            plt.axis('equal')
            plt.title(f'Non Dimn temp distribution at t = {t_mark}s')
            plt.show()
            t_plotted[t_mark] = True

    # Checking the tolerance condition
    if np.max(np.abs(Phi - Phi_old)) < tol:
        steady = True
        print(f"Steady state reached at t = {t_cur:.2f}s")
        plt.figure(figsize=(6, 5))
        cp = plt.contourf(X, Y, Phi, 100, cmap='jet')
        plt.colorbar(cp)
        plt.axis('equal')
        plt.title(f'Steady State Non Dimn temp distribution(t =
{t_cur:.2f}s)')
        plt.show()

```